

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module-4.2]

DISTRIBUTED ALGORITHMS - AN OVERVIEW

TEXTBOOK REFERENCE

- These slides are based on the textbooks:
 - “Elements of Distributed Computing”, Vijay K. Garg
 - “Distributed Systems - Concepts and Design” Coulouris et al.

DISTRIBUTED ALGORITHMS

- Main challenges
 - no shared clock, no shared memory, failures
- General idea
 - using logical clocks and *happened before* relation to define an order between (related) events
 - assumption for most algorithms:
 - asynchronous systems, but no failures cioè se io mando un messaggio, prima o poi arriva però non ho idea quando
 - finite but not known a priori delivery time
 - synchronous systems when dealing with failures
 - timeouts usare la sincronia per gestire i failure: se uso l'asincronia non posso dire "quanto tempo devo aspettare per avere un ack" perchè "prima o poi arriverà, ma non so quando". Quindi, con sincronia, assumo che ci sia stato un fallimento facendo scattare un timeout

MAIN CATEGORIES di algoritmi distribuiti

- **Events ordering (partial)**
 - Logical clocks, vector clocks
 - discussed in previous module
 - used by all other algorithms
- **States Observation**
 - Global snapshots (per scattare degli snapshot di ogni stato locale di ogni componente del sistema per costruire uno stato globale del sistema)
- **Coordination & Agreement**
 - Distributed Mutual Exclusion
 - Election
 - Multicast communication
 - Consensus
- **Distributed Transaction and Concurrency Control**
- **Replication & Fault-Tolerant Services**

Obiettivo di questa lezione: capire qual'è il ragionamento di fondo da assumere per osservare questi algoritmi e per crearne di nuovi

IN THIS MODULE

- **Mutual exclusion**
 - centralised algorithm
 - Ricart-Agrawala decentralised algorithm
- **Leader election**
 - Chang-Roberts algorithm
- **Message ordering**
 - causal message ordering
 - total message ordering
- **Consensus**
 - consensus problem definition
- **Global snapshots**
 - Chandy-Lamport algorithm

MUTUAL EXCLUSION

MUTUAL EXCLUSION & CRITICAL SECTION

- Analogous to the problem in the concurrent case
 - here processes are distributed
- Critical section properties
 - **safety**
 - two processes should not have the permission to be in their CS simultaneously
 - **liveness**
 - every request for the CS is eventually granted
 - **fairness**
 - Different requests must be granted in the order they are made

Qui, non posso usare Mutex/Semafori. Pensare ad un algoritmo che si basa sullo scambio di messaggi asincrono, con le assunzioni che abbiamo fatto, che garantisca che riesca ad implementare la sezione critica

CENTRALIZED ALGORITHM

- There is a *coordinator* giving the token →
- Key point: **fairness**
 - requests should be given access according the **happened before relation**
 - given s, t = two local states (of different processes), if $s \Rightarrow t$ then request done in s should be served before request done in t
 - even if a request done in t arrives to the coordinator before a request done in s , the coordinator must understand that there is a pending request from the process P related to s
 - the coordinator can delay the receiving of the message t until all requests before t are received
- when communicating with the coordinator, a process includes its knowledge about the requests done so far by all processes, so that if $s \rightarrow t$, then t has the information about s
 - information about requests known at the state s are captured using a vector $s.v$, where $s.v[j]$ represents the number of requests made by P_j that causally precede the state s

Figura centrale con gli n processi a cui fanno riferimento (chiunque voglia entrare in sezione critica, deve mandare un messaggio al coordinatore con la richiesta e per accedere deve ricevere una risposta dal coordinatore con il token)



vogliamo servire le richieste che arrivano nell'ordine definito dalla relazione Happened-before

(soluzione: etichettiamo tutte le richieste con un Vector clock in modo che il coordinatore possa ordinare le richieste secondo vector clock)

CENTRALIZED ALGORITHM

- Idea
 - processes cooperate by piggybacking vector clock v on all outgoing msgs
 - the coordinator process delays a request t until all requests before t are received
- Data structures on the coordinator side P_0
 - **reqlist**
 - list of requests received that have not been fulfilled yet
 - a request w is **eligible** in this list *if there is no request that happened before w and has not been fulfilled yet*
 - **reqdone**
 - vector such that $\text{reqdone}[i] = \text{number of requests made by } P_i \text{ that have been fulfilled}$
non è un Vector Clock, ma ho lo stesso principio (quante volte i processi sono entrati in CS)

Ci si propaga, tra Client process, i numeri di richieste fatte al Coordinatore: in modo tale che il Coordinatore sa se ci sono altre richieste che devono ancora arrivare che andrebbero soddisfatte prima rispetto a richieste che sono arrivate ma richieste dopo (Esempio: P1 ha fatto una richiesta al Coordinatore prima di P2, ma arriva prima quella di P2, ma P1 aveva detto a P2 che aveva già fatto una richiesta prima della richiesta di P2, quindi P2 aggiorna il suo Vettore di richieste totali fatte in modo tale che il Coordinatore sa che deve aspettare la richiesta di P1 prima di soddisfare la richiesta di P2. Quindi, P2 manderà, oltre alla richiesta, il suo vettore dei conteggi delle richieste totali fatte sue e di quelle delle altri, in base alla sua conoscenza)

Client process P_i

All'arrivo di messaggio di P2: [1,1] (non viene servita perchè C ha il contatore a [0,0] e quindi deve ancora ricevere la richiesta di P1 che è stata fatta prima di P2). All'arrivo di messaggio di P1, [1,0]: allora viene servita perchè di P2, C non ha ancora servito niente, mentre la prima di P1 è proprio questa

```
P[i]::
var
  v: array[1..N] of integer := [0]
  inCS: boolean := false

To request:
  v[i] := v[i] + 1
  send (req, v) to P[0]

Upon receive(token) from P[0]:
  inCS := true

To release:
  send token to P[0]
  inCS := false

Upon receive(u):
  v := max(v,u.v)
```

Coordinator

Un processo propaga, ogni volta che comunica con un altro processo, la conoscenza del proprio stato ed anche la sua conoscenza di quello che hanno fatto gli altri processi (Vettore di richieste fatte in totale)

```
P[0]::
var
  reqDone: array[1..N] of integer := [0]
  reqList: list of (pid, reqvector) := empty
  haveToken: boolean := true

Upon receive(u,req):
  append(reqList,u)
  if (haveToken)
    checkReq()

Upon receive(u,token):
  haveToken := true
  checkReq()

checkReq:
  eligible := {w in reqList |
    for each j, if j != w.p: w.v[j] == reqDone[j] }
    if j == w.p: w.v[j] == reqDone[j] + 1 }
  if (eligible != {})
    w := first(eligible)
    delete(reqList,w)
    reqDone[w.p] := reqDone[w.p] + 1
    send token to P[w.p]
    haveToken := false
```

A request w is *eligible* if w.v is at most reqdone, i.e. there is no request that happened before w and has not been fulfilled yet

Se non ci fosse state interazione tra P2 e P1, P2 non sapeva che P1 aveva mandato richiesta e quindi avrebbe mandato un vettore [0,1] (non c'è Relazione Happened-before: evento concorrente)

EXPLANATION

- Client process P_i

- To request the token, il client incrementa la propria componente nel vettore v the client simply implements its component in v vector and sends the request with its vector v to the coordinator P_0
- On receiving the token, the client proc. can enter the critical section
- To release the critical section, it simply returns the token to P_0
- To ensure fairness, processes cooperate by piggybacking v on all messaggi in uscita outgoing messages
- When a process receives a message, it updates its v vector tra il proprio valore precedente e il vettore ricevuto con il messaggio with component-wise calcolando il massimo elemento per elemento max of its old value and the vector received with the msg

- Coordinator coordinator P_0

- It maintains a vector *reqdone* with the interpretation that *reqdone[i]* equals the number of requests made by P_i that have been fulfilled
- It also maintains *reqlist*, which is the list of requests received by P_0 that have not fulfilled yet
- On receiving a request
 - P_0 appends it to *reqlist*
 - If it has the token, then it checks for **eligible** requests and sends the token to one of the eligible processes

A request w is *eligible* if $w.v$ is at most redone, that is: no requests that happened before w and had not been fulfilled yet

↳ Una richiesta w è ammissibile se $w.v$ è al massimo in fase di rielaborazione, ovvero: non ci sono richieste avvenute prima di w che non siano state ancora soddisfatte.

DECENTRALIZED SOLUTION: RICART AGRAWALA'S ALGORITHM

Se il processo è in CS e riceve la richiesta da un altro processo che vuole entrare in CS, metto la richiesta del processo in una lista di pending request (ci possono essere altre richieste da altri processi) e non risponde subito (come faceva il Coordinatore, si ritardano i messaggi a quando ha finito la sua CS: quando ha finito, manda che ha finito a tutti i processi)

- **Idea:** Senza coordinatore: per entrare in CS, devo chiedere a tutti gli altri processi il permesso di entrare in CS (entro se ricevo OK da tutti i processi)
 - to request a resource, a process sends a timestamped message to all processes
 - on receiving a request from any other process, a process ^{ricevente la richiesta} sends an OK message ^{e non è in CS} if either the process is not interested in entering in CS or if its own request has a higher timestamp value. Otherwise that process is kept in a pending queue
 - To release a resource, a process sends OK to all the processes in the pending queue non viene scelto uno perché ci possono essere più richieste (ma così non si ha una violazione della mutua esclusione? No perché il processo per entrare deve ricevere OK da tutti gli altri processi)
 - A process is granted the resource when it has requested the resource and it has received the OK message from every other process in response to its request msg

G.Ricart, A.K. Agrawala. *An optimal algorithm for mutual exclusion in computer networks*. Communication of ACM, 24, 1981

Se voleva entrare in CS (il processo ricevente la richiesta) ma il suo timestamp/priorità è più alto/bassa del richiedente (richiesta Happened-before di quella del ricevente) allora mando OK; Se ho priorità più alta io, allora lo metto in Pending queue

```

P[i]::

var
    pendingQ: list of process ids initially null
    myts: integer timestamp initially infinity
    numOK: integer initially 0

request:
    myts := logical_clock
    send req with myts to all other processes
    numOK := 0

receive(u, req):
    if (u.myts < myts)
        send OK to process P[u.pid]
    else
        append(pendingQ, u.pid)

receive(u, OK):
    numOK := numOK + 1
    if (numOK == N-1)
        enter_CS

release:
    myts := infinity
    for each pid in pendingQ
        send OK to P[pid]
    empty(pendingQ)

```

- Number of messages: $2(N-1)$
- It works with channels not necessarily FIFO

Si potrebbe risolverlo con una tipologia ad anello (definisco un ordine nello scambio di messaggi tra processi)

LEADER ELECTION

Come faccio a definire chi tra processi è il
Coordinatore in modo dinamico (e.g. in caso vada
giù il Coordinatore)? Tramite Leader Election

LEADER ELECTION

Si basano sulla definizione di una topologia ad anello per fare in modo di scegliere il Leader

- Choosing one process among a set of N processes
- Based on superimposing a *logical ring topology* on the underlying network to execute control function
- Widely used strategy to choose coordinators in centralized algorithms

CHANG-ROBERTS ALGORITHM

Avere un criterio che permette di selezionare il Leader (in questo caso, chi ha il PID maggiore)

- **Assumption** L'aspetto del "se ci sono nodi giù nel mentre che facciamo Leader election" è ortogonale, cioè non preso in considerazione dall'algoritmo
 - each process has a unique Pid
- **Idea**
 - the process with the max Pid is elected
 - one or more processes can spontaneously wake up and initiate an election
 - Uno o più processi si rendono conto che c'è bisogno di eleggere un nuovo Leader (e.g. Coordinator va giù)
 - a process
 - sends an election msg along with its identifier to its left, if it has not seen any msg with a higher identifier than its own
 - it forwards any message that has an identifier greater than its own, ~~otherwise it swallows the message~~ altrimenti ignora
 - if a process receives its own message => it declares itself as a leader and propagate a leader msg
 - è il Leader data la topologia della rete logica tra processi

Questo algoritmo funziona anche quando più processi fanno partire contemporaneamente più leader election, comunque arrivando ad un Leader

E.J.H. Chang, R. Roberts. *An improved algorithm for decentralized extrema-finding in circular configurations of processes*. Communications of ACM, 22(5):281-283, 1979.


```

P[i]::

var
    myid: integer
    awake: boolean := false
    leaderId: integer := null

To initiate election:
    send (election,myid) to P[i-1]
    awake := true

Upon receiving a message (election,j):
    if (j > myid)
        send (election,j) to P[i-1]
    else if (j == myid)
        send (leader,myid) to P[i-1]
    else if ((j < myid) and !awake)
        send (election, myid) to P[i-1]
    awake := true

Upon receiving a message (leader,j):
    leaderId := j
    if (j != myid)
        send (leader,j) to P[i-1]

```

- Number of messages exchanged
 - worst case: election msgs are $2N-1$ election + N leader msgs

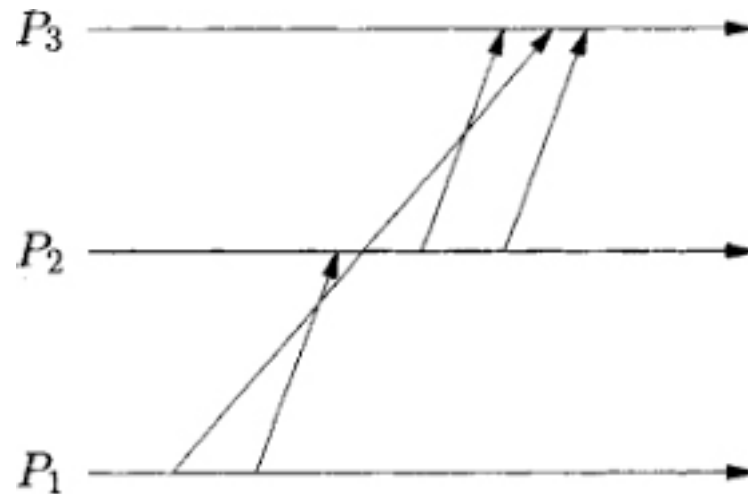
MESSAGE ORDERING

CAUSAL MESSAGE ORDERING

- Fully async computation => no restrictions in msg ordering
 - max non-determinism, max concurrency
- Sometimes it is useful to reduce non-determinism by restricting the possible message ordering in a system
- From FIFO-ordered computation to **causal ordering**
 - given 2 msgs m_1, m_2 such that $\text{send}(m_1) \rightarrow \text{send}(m_2)$, then $\text{rec}(m_1) \rightarrow \text{rec}(m_2)$
 - no overtaking

Se la send di m_1 è successo prima della send di m_2 , il ricevente dei due messaggi deve ricevere prima m_1 poi m_2

l'ordinamento causale dei messaggi permette di astrarre dai ritardi fisici e dal non-determinismo della rete, garantendo che l'applicazione percepisca i messaggi rispettando la relazione Happened-before indipendentemente dall'ordine reale con cui i pacchetti viaggiano sui canali fisici.



A FIFO computation that is not causally ordered
(CHAPTER 16, Garg)

CAUSAL MESSAGE ORDERING

- Algorithm ^{che garantisce} ensuring causal ordering - Idea:
 - each process ^(P_i) maintains a **matrix m** of integers (extension of vector clocks) matrice che tiene conto di quanti messaggi hanno inviato i vari processi a tutti gli altri processi (matrice costruita in base alla conoscenza del processo che ha quella matrice)
 - $s.m[j, k]$ = number of msgs sent by a process P_j to P_k, as secondo quanto noto a P_i nello stato s. known by P_i in state s riga j della matrice: numero di messaggi mandati da un processo P_j a tutti gli altri processi
riga k della matrice: numero di messaggi mandati ad un processo P_k da tutti gli altri processi
 - ~~whenever a msg is sent from P_i to P_j,~~ Ogni volta che
 - the matrix m is piggybacked with the msg Ogni processo quando invia un messaggio, aggiorna la matrice della conoscenza che ha relativamente quali sono i messaggi che sono stati scambiati e manda con il messaggio la matrice
 - $m[i, j]$ is incremented ~~to reflect that one more msg has~~ per riflettere l'invio di un ulteriore been sent from P_i to P_j
 - ~~when a msg is received by the communicating system, first it is~~ Quando un messaggio viene ricevuto dal sistema di comunicazione ~~checked for eligibility before delivery to P_i~~ prima della consegna a P_i ne viene verificata l'idoneità (eligibility)
 - a msg u is **eligible** to be received in state s by P_i when the number of msgs sent from any process P_k to P_i as reported by $u.m$ is less or equal to the number recorded in $s.m$
 - if it is not eligible, the message is buffered until it can be received Il processo ricevente controlla la colonna della sua matrice e quella ricevente riguardante il suo processo; se ogni cella della colonna della matrice del messaggio è maggiore rispetto alla colonna della matrice del processo ricevente, allora quel messaggio non viene ricevuto e si aspetta che arrivino tutti i messaggi che ci sono prima (rispetto Happened-before)

Esempio: c'è un messaggio di P1 che è ancora pendente e che non è ancora arrivato a P3, ma è arrivato prima il messaggio di P2 che è stato inviato dopo il messaggio di P1 verso P3. Quindi, il messaggio di P2 non viene ricevuto da P3 fino a che verso P3 non arrivino tutti i messaggi mandati prima di P2 seguendo la relazione Happened-before (il messaggio di P2 viene messo in un Buffer del sistema di comunicazione). NOTA: se P1 non avesse "parlato" con P2, P3 non sarebbe stato in grado di capire che il messaggio di P2 era stato inviato dopo quello di P1 (considerati eventi concorrenti: non c'è relazione Happened-before)

Pi::

var

/* local matrix clock m */

m: array[1..N,1..N] of integer := 0

For every msg sent (from process Pi) to process Pj:

- $m[i,j] := m[i,j] + 1$
- piggyback the matrix m along with the msg

For every msg received (by Pi) from process Pj, with matrix m':

the msg is eligible if

- $\forall k$, if $k \neq j$: $m'[k,i] == m[k,i]$, and
if $k == j$: $m'[k,j] == m[k,j] + 1$

if the message is not eligible

then delay the msg (put the msg in a queue)

else

update m with m',

receive the msg and

check for other possible eligible messages

FURTHER EXPLORATIONS

Come se fosse Message Passing Sincrono

Altri algoritmi di Message ordering

- • Synchronous / Total order
 - stronger than causal ordering
 - setting up a coordination such that all messages can be treated as logically instantaneous (come se la durata dell'invio/ricezione fosse istantanea)
- • Total order for multicast messages
 - in the case that a message is sent to multiple processes, all messages are delivered in the same order at all processes
 - for all messages x and y and all processes P and Q , if x is received at process P before y , then y is not received before x at Q

SYNCHRONIZERS

Fanno in modo che l'invio e la ricezione dei messaggi avvenga per round di sincronizzazione (cioè nei momenti in cui tutti possono mandare messaggio e riceverlo etichettando il tempo di ricezione con il round). Da usare quando vogliamo un ordinamento totale dei messaggi, con elaborazione dei messaggi sincrona

- The design of distributed algorithms is easier *if we assume that the underlying network is synchronous rather than asynchronous*

- methods to simulate sync network over async
 - transmission time ~ 0
- assuming absence of failures or known max delay

Proprietà fondamentale: Un sistema basato su impulsi garantisce che qualsiasi messaggio inviato durante l'impulso i venga ricevuto dal destinatario all'interno dello stesso impulso i . Obiettivo: Creare l'illusione che tutti i processi avanzino "in parallelo passo dopo passo", esattamente come avverrebbe in un sistema sincro perfetto scandito da un clock comune.

- Mechanisms: **pulses** and **synchronizers**

- a pulse is a counter at each process with the property that any msg sent in a pulse i is received at pulse i
- a synchronizer is a mechanism that indicates to a process when it can generate a pulse

Il sincronizzatore è il meccanismo che dice al singolo processo quando è sicuro incrementare il contatore dell'impulso e passare alla fase successiva.

- Complexity

- message complexity
 - additional number of messages required by the synchronizer to simulate a synch algorithm on top of an async network
- time complexity
 - number of time units required to simulate one pulse where a time unit is defined as the time required for an async msg

SYNCHRONIZERS

- Idea
 - every process sends exactly one msg to all neighbors in each pulse
 - if a process is in round i and it gets a msg with pulse $i+1$, then it buffers that msg and waits for the msg with pulse number i

```
Pj::  
  
var  
    pulse: integer := 0  
  
round i:  
    pulse := pulse + 1  
    simulate the round i of the synch algorithm  
    send msg to all neighbors with pulse  
    wait for exactly one msg from each neighbor with (pulse = i)
```


GLOBAL STATE OBSERVATION

GLOBAL STATE

Obiettivo: catturare lo stato globale del sistema (esempio: il sistema è in Deadlock? catturare lo stato di tutti i processi che compongono il sistema).

- Capturing the *global state* of a distributed system from a single process

- Def. **global state** è uno stato consistente che è dato dall'insieme di stati locali che sono tutti concorrenti tra di loro (cioè nessuno stato locale happened-before dell'altro perché se uno stato è Happened-before di un altro allora non fanno parte dello stesso stato/snapshot globale presente: fanno parte di due snapshot globali diversi)

- *is a set of local states that are all concurrent with each other (according to the happened before model)*

- for many applications, it is enough to capture a global state that happened in the past, not the current

- An algorithm that captures a global state is called *global snapshot algorithm*

- Challenge per farlo in modo dinamico ed asincrono, Chandy-Lamport introduce il concetto di Taglio consistente

- a *consistent* global state is *not* simply the product of local states

- def. **CONSISTENT CUT**

L'evento e' corrispondente all'invio di quel medesimo messaggio msg.

- a global state S where $\forall \text{ event } e \mid e = \text{receive}(\text{msg}),$
and $e' = \text{send}(\text{msg}), e' \rightarrow e \Rightarrow e' \in S$

t.c

Un qualsiasi evento e che consiste nella ricezione di un messaggio msg.

Happened-before

Se la ricezione e fa parte della fotografia, allora anche l'invio e' deve fare parte della stessa fotografia.

Una sfida riguarda il fatto che gli stati locali devono rimanere quelli (dopo aver mandato il proprio stato) e non aggiornarsi mentre si cattura lo stato globale. Una prima soluzione riguarda mandare un messaggio di inizio cattura stato globale, ricevo ack e quindi i processi si fermano (quindi non ci sono messaggi nei canali) e si prende lo stato locale di ogni processo e si ottiene lo stato globale. La vera sfida è quello di catturare lo stato globale in modo dinamico ed asincrono (cioè senza bloccare i processi): fatto da Chandy-Lamport Snapshot Algorithm

CONSISTENT AND INCONSISTENT CUT

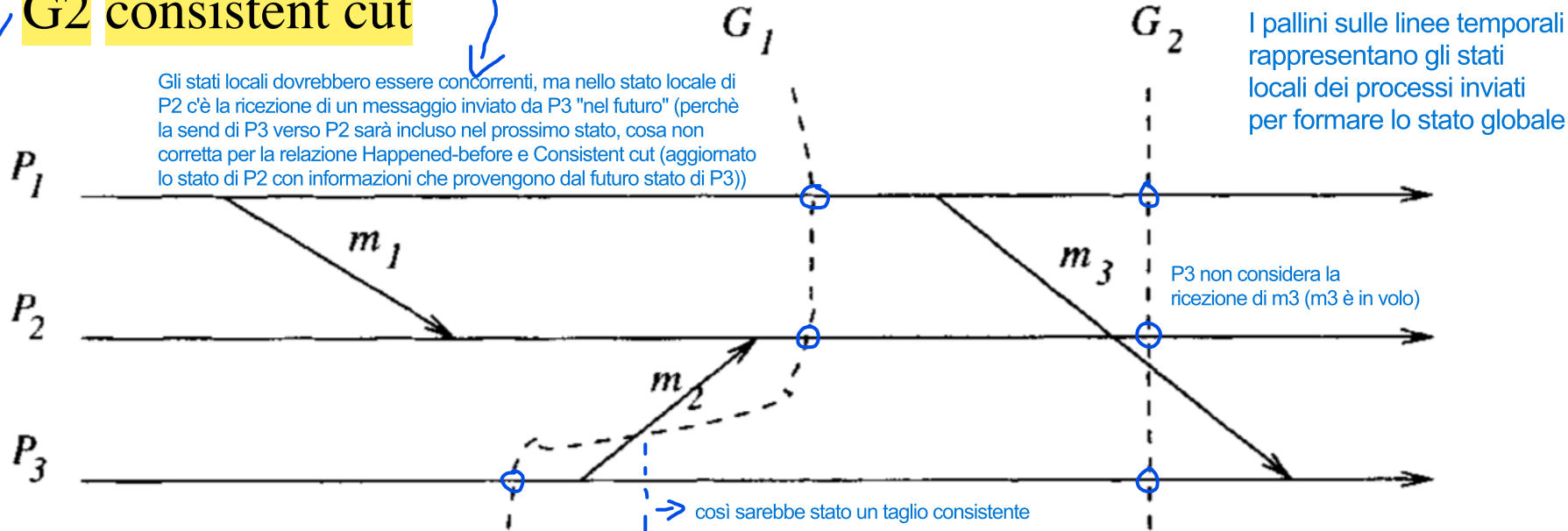
Esempio di Taglio Inconsistente

Immagina una fotografia dello stato di due persone (P1 e P2) in cui: P2 risulta aver ricevuto una lettera da P1 (e' in S), ma nella fotografia di P1, lui non ha ancora spedito quella lettera (e' non in S). Questo crea una contraddizione paradossale: il sistema registra un effetto (ricezione) senza la sua causa (invio).

G1 inconsistent cut

G2 consistent cut

Gli stati locali dovrebbero essere concorrenti, ma nello stato locale di P2 c'è la ricezione di un messaggio inviato da P3 "nel futuro" (perché la send di P3 verso P2 sarà incluso nel prossimo stato, cosa non corretta per la relazione Happened-before e Consistent cut (aggiornato lo stato di P2 con informazioni che provengono dal futuro stato di P3))



I pallini sulle linee temporali rappresentano gli stati locali dei processi inviati per formare lo stato globale

P3 non considera la ricezione di m_3 (m_3 è in volo)

→ così sarebbe stato un taglio consistente

Esempio di Taglio Consistente

Un taglio è consistente se rispetta la relazione Happened-before: se un messaggio è stato ricevuto, nello snapshot risulta anche inviato (e in S e e' in S). Se un messaggio è stato inviato ma non ancora ricevuto, l'invio fa parte dello snapshot (e' in S), mentre la ricezione no (e non in S). Questo è perfettamente valido: significa semplicemente che il messaggio era in transito (in volo sulla rete) quando è stata scattata la foto.

(Fig. from "Elements of Distributed Computing", Vijay K. Garg)

CHANDY-LAMPORT SNAPSHOT ALGORITHM

Lo stato globale viene catturato senza bloccare i processi e cattura sia gli stati locali dei processi sia gli stati dei canali (in cui potrebbero esserci dei messaggi in volo) perché lo snapshot deve essere fatto in modo asincrono e dinamico

- Algorithm to take a global consistent snapshot of a distributed system, ^{calcolando} by computing a consistent cut

- Challenges

- ensuring that local states are *concurrent*
- doing the snapshot *without blocking* the processes
- capturing the state of the channels

- *marker mechanism*

Messaggi di controllo che attivano la registrazione dello snapshot (i marker coordinano la cattura dello snapshot globale senza bloccare il sistema)

- Assumptions ^{i processi comunicano attraverso dei canali. I canali rispettano politica FIFO e messaggi ordinati}

└ all channels are unidirectional and FIFO



I canali sono affidabili (non ci sono messaggi persi)

I processi non falliscono durante lo snapshot

La rete è fortemente connessa (ogni processo può comunicare con tutti gli altri)

C.K. Chandy, L. Lamport. *Distributed Snapshots: Determining global states of distributed systems*. ACM Transaction on Computer Systems. 3(1):63-75. Feb. 1985

CHANDY-LAMPORT SNAPSHOT ALGORITHM

Idea

L'algoritmo sfrutta l'astrazione concettuale della colorazione dei processi e la proprietà dei canali FIFO per separare il passato (prima dello snapshot) dal futuro (dopo lo snapshot) senza dover arrestare l'esecuzione del sistema.

- each process ^{possiede una} color variable (*red|white*): the computed global snapshot corresponds to the state of the system just before the processes turn to red
- all processes are initially *white* non stanno scattando la foto
- after recording a local state, a process turns to red
 - the state of a local process is simply the state just before it turned to red

To record the state of the channels:

- once a process turns to red, it sends a marker msg ^{lungo tutti i suoi} ~~along all its~~ ^{canali uscenti} ~~outgoing channels~~ before it sends out any other msg
- a process is required to turn red on receiving a marker if it has not already done so. ^{Poiché i canali sono FIFO:} ~~Having FIFO channels:~~ Poiché i canali sono FIFO (First-In, First-Out), qualsiasi messaggio inviato da un nodo prima del marker arriverà a destinazione prima del marker stesso.

- ^{Nessun processo bianco riceve mai} ~~no white process ever receives~~ a message sent by a red process

- this ensures that local states are ^{tra di loro} ~~mutually~~ concurrent

- ^{è definito dai} ~~the state of the channels is given by~~ messages sent by a white process and received by a red process

- ^{cioè} msgs that were in transit when the snapshot is taken

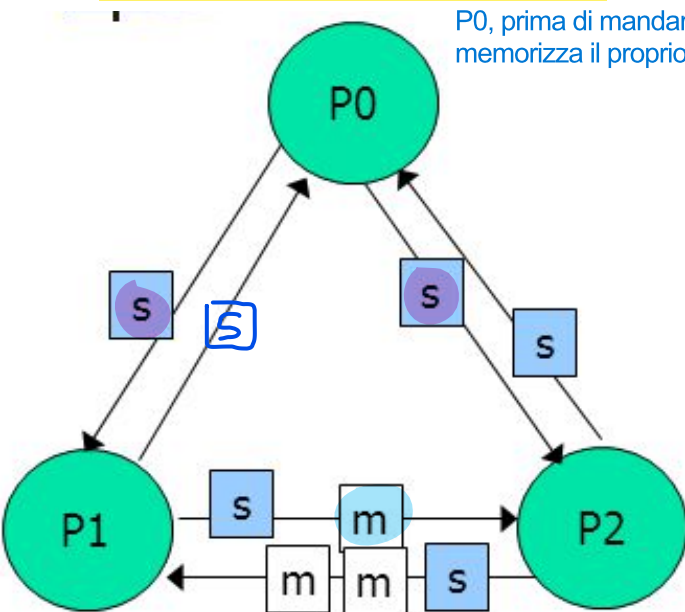
- Stato Bianco rappresenta l'esecuzione del sistema prima che lo snapshot abbia luogo.
- Stato Rosso rappresenta l'esecuzione del sistema dopo che lo snapshot è stato avviato.
- Lo snapshot globale registrato è esattamente la combinazione degli stati locali di ciascun nodo presi un istante prima del loro passaggio da bianco a rosso.

Questo garantisce un'invariante fondamentale: un processo bianco non riceverà mai un messaggio spedito da un processo diventato rosso. Se ciò accadesse, l'effetto (ricezione) sarebbe stato registrato nel passato (bianco), mentre la causa (invio) farebbe parte del futuro (rosso), creando un inconsistent cut.

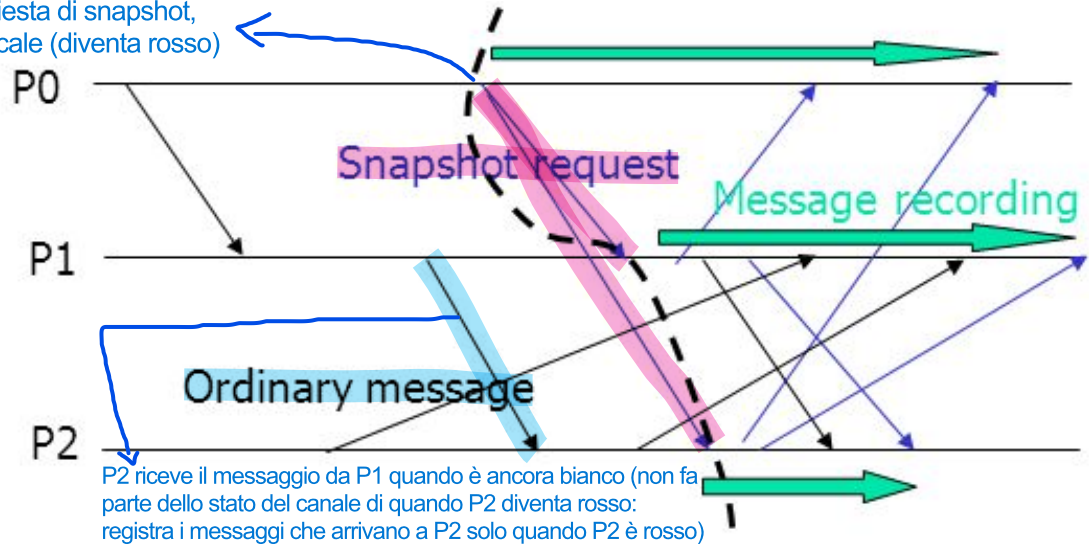
CHANDY-LAMPORT SNAPSHOT ALGORITHM

Inizialmente, i processi sono bianchi (non stanno scattando la foto); quando ricevono la richiesta di snapshot, allora iniziano a catturare lo snapshot

Un processo, dopo aver ricevuto la richiesta di snapshot, da lì in avanti, inizia a tracciare tutti i messaggi che riceve dai canali in ingresso



P0, prima di mandare la richiesta di snapshot, memorizza il proprio stato locale (diventa rosso)



Chi e come determinare lo stato dei canali (cioè i messaggi in volo che costituiscono lo stato dei canali nel taglio consistente)?

- A process, that wants to take a snapshot, sends a snapshot request to the others.
- Each process records its state upon receiving the first snapshot request. (diventano rossi, sapendo della cattura)
- Each process keep recording the messages until receiving a snapshot request from each of the other process except the one that has originally initiated a snapshot. (continua a registrare da tutti gli altri processi fino a quando non riceve eccetto quello che ha avviato originariamente)

Si diventa da bianco a rosso (primo messaggio di snapshot) e si inizia a tracciare/memorizzare tutti i messaggi in entrata. Al secondo messaggio di snapshot in entrata, si ferma il tracciamento (si diventa bianchi)

(From <http://slideplayer.com/slide/1506898/>)

ricevuta la richiesta di snapshot, i processi inviano a loro volta la richiesta di snapshot ai processi con cui sono collegati

```

Pi::

var
  color: {white,red} := white
  // assume k incoming channels
  chan: array[1..k] of queues of msg := empty // state of the channels to be recorded
  closed: array[1..k] of boolean := [false]

turn_red() enabled if (color = white):
  save_local_state
  color := red
  send maker to all neighbors

Upon receive(marker) on incoming channel j:
  if (color = white)
    turn_red()
  closed[j] := true

Upon receive(program_message) on incoming channel j:
  if (color = red and not closed[j])
    append(chan[j], program_message)

```

FURTHER ISSUES

Usa l'algoritmo per:

- Observing *Global Predicates* se vale una certa proprietà nel sistema
 - linear, regular, conjunctive, channel predicates
- Applications
 - deadlock detection
 - termination detection
- Computing *Global Functions*

Spiegato in modo marginale

CONSENSUS

CONSENSUS

- *Consensus* is a fundamental problem in distributed computing
finding agreement among distributed processes about the value of some property, about some action to do, etc.
- Consensus algorithms are pervasively used in real-world systems
 - transactions in DB (whether to commit a transaction), leader election (agreeing on the identity of a leader), state machine replication, atomic broadcasts, clock synchronization, blockchain, etc

CONSENSUS IN ASYNCHRONOUS NET

- In **asynchronous** network, in the presence of even a single unannounced process death/failure, the consensus problem is *impossible to solve*
 - **FLP result** (Fischer, Lynch, Patterson)

Given the general properties of consensus, all problems are impossible to solve in the case of process failure in the case of fully async networks..

Fischer, Lynch, Patterson. *Impossibility of distributed consensus with one faulty process.* Journal of ACM, 32(2). April 1985

CONSENSUS UNDER SYNCHRONY

- Synchronous systems
 - an **upper bound** can be assumed on the message delay and on the duration of actions performed by processes
- 4 tipi di modelli di failure
~~Type of faults, models:~~
 - **crash** un nodo va giù e gli altri nodi sanno che è andato giù (va giù e non torna su)
 - crash model => a fault corresponds to a processor halting
 - **crash+link**
 - in this model either a processor can crash or a link may go down. If the link goes down, it stays down
 - **omission**
 - a process fails either inviando solo rispetto ai messaggi prescritti dall'algoritmo by sending a proper subset of messages that it is required to send by the algorithm or ricevendo che gli sono stati inviati by receiving only a proper subset of messages that have been sent to it
 - **byzantine failure** il nodo dice cose non vere esibendo un
 - a processor fails by exhibiting arbitrary behaviour

DEFINITION OF THE PROBLEM

Il problema è se ci sono dei fallimenti durante l'esecuzione dell'algoritmo di consenso (si complica l'algo): l'algo diventa banale se non ci sono fallimenti

→ Simile al problema del Massimo-Minimo nel modulo del Message Passing

- **Definition of the problem** Raggiungere una decisione insieme (scegliere un valore secondo un criterio comune (e.g. valore che la maggior parte ha in comune in base ai voti))

- To reach consensus, every process P_i begins in the *undecided state* and proposes a single value v_i , drawn from a set D ($i = 1..N$)
- The processes communicate with one another, exchanging values
- Each process finally sets a *decision variable* d_i with the decided value
 - in doing so, it enters *the decided state*, in which it may no longer change d_i

- **Requirements**

- **Termination.** Eventually each *correct* process sets its decision variable Ogni processo corretto (che non ha avuto fallimenti) definisce la decision variable alla fine
- **Agreement.** The decision value of all correct processes is the same: if P_i and P_j are correct and have entered the decided state, then $d_i = d_j$ La decisione di tutti i processi corretti deve essere la stessa (il criterio è indifferente, basta che si converge ad un'unica decisione)
- **Integrity.** If the correct processes all proposed the same value, then any correct process in the *decided state* has chosen that value (i.e. don't change idea..) Arrivati alla decisione finale, non si può cambiare idea (devo mantenere il valore a cui si è fatta convergenza)

BASIC ALGORITHM

Se ci sono al massimo f nodi che possono andare in crash e noi eseguiamo $f+1$ round, è matematicamente garantito che esista almeno un round in cui NESSUN nodo va in crash (oppure un round in cui nessun nodo va in crash mentre sta inviando messaggi).

- Idea

- each process maintains V , the set of values it knows that have been proposed by processors in the system
 - initially a process knows only its own value
- given the V set, it decides using some criteria / decision strategy
 - that should be the same for all processes

- Algorithm based on *rounds*

non ci interessa quando i processi andranno giù: applicandolo $f+1$ volte siamo coperti con al massimo f processi in failure

Se voglio resistere ad un numero di processi che possono andare in failure pari ad f , allora per resistere ad f failure mi basta fare $f+1$ round (cioè inviare il proprio valore $f+1$ volte)

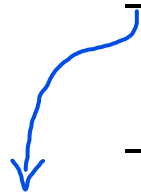
- each correct process executes **$f + 1$ rounds**
 - where $f = \text{max number of processors that can fail}$
- in each round

numero massimo di processori che possono subire un guasto (crash)

- a process sends to all other processes values from V that it has not sent before
- a process P_i receives the value sent by P_j
 - synchrony assumption: waiting some fixed predetermined time, after that it assumes that P_j crashed

manda sia il suo valore sia i valori che quel processo sa da altri processi

- It works for in the case of no byzantine behaviours



Lo faccio $f+1$ volte, così nel caso in cui ci sono uno dei tre failure dei primi 3 modelli, l'algoritmo è robusto



Pi::

var

V: set of values := { v[i] }

for k := 1 to f + 1 do

send { v in V | Pi has not already sent v } to all

receive Sj from all process Pj, j != i

V := V U Sj

y := decide(V)

“*decide*” denotes the criteria (function) adopted to decide the value. The important thing here is that all processes would use the same criteria.

- Number of messages required: $O((f+1)N^2)$

CONSENSUS UNDER BYZANTINE FAULTS

- Case in which faulty processes can have arbitrary behaviour
 - sending wrong values
- **Byzantine General Agreement (BGA) problem**
 - designing a protocol for which the *loyal generals* (correct processes) can coordinate their actions (deciding to attack or retreat altogether) in spite of the treacherous processes (faulty)
- **Result (theorem)** Per avere un protocollo di consenso resiliente (quindi si raggiunge consenso), bisogna avere, rispetto ai processi in failure, maggiore di tre volte di più processi corretti (cioè $N > 3F$)
 - *There is no f -resilient protocol for BGA if $N \leq 3f$*
 - where N = total number of ^{correct} processes, f = number of faulty processes

L. Lamport, R. Shostak, M. Pease. *The Byzantine generals problem*. ACM Transaction on Programming Languages and Systems, 4(3):382-401, 1982

CONSENSUS UNDER BYZANTINE FAULTS

- Idea

- $f+1$ rounds, f number of faulty processors
- rotating coordinator (called **king**)
 - at least 1 round has a king which is not faulty
- Processor P_i assumes to be the coordinator or king for round k
- 2 stages

Poiché nel sistema ci sono al massimo f nodi guasti/bizantini, eseguendo $f+1$ round si ha la garanzia matematica che almeno un round avrà come coordinatore (King) un nodo leale e corretto.

- first stage: each processor exchanges its value to all other processors and based on V it determines its estimate
 - *myvalue* variable
- second stage: it receives the value from the coordinator
 - if no value received (coordinator failed), then it assumes null
 - then it decides if keeping its own value or the king value
 - » the decision is based on the multiplicity of *myvalue* in the vector V
 - » if V has more than $N/2+f$ copies of *myvalue*, then *myvalue* is chosen for $V[i]$; otherwise kingvalue is used

Calcola il valore di maggioranza all'interno del vettore V e lo salva nella variabile locale *myvalue*.

Ogni processo decide se mantenere il proprio *myvalue* o adottare il kingvalue basandosi sulla frequenza di *myvalue* in V :
- Se V contiene più di $(N/2) + f$ copie di *myvalue*: Il processo mantiene il proprio *myvalue*.
- Altrimenti: Il processo sovrascrive il proprio valore adottando il kingvalue inviato dal re.

```

Pi::

var
  V: array[1..N] of values := {v[i] = x, v[j] = not defined}

for k := 1 to f + 1 do
  first phase:
    send V[i] to all other processors
    set V[j] (j != i) to the value received from Pj
    myvalue := majority in the vector V (not defined, if no majority)

  second phase:
    if (k == i) // I'm the king in this round
      send myvalue to all other processors
      receive king value from Pk
    if V vector has more than  $N/2 + f$  copies of my values
      V[i] := myvalue
    else
      V[i] := kingvalue

y := V[i]

```

FURTHER EXPLORATIONS - PAXOS

- **Paxos**

- family of protocols per il raggiungimento del consenso in una rete di ~~for solving consensus in a network of~~ processori inaffidabili ~~unreliable processors~~ (e.g. per avere consenso tra repliche di un stesso database)
- including a spectrum of trade-offs between the number of processors, number of message delays before learning the agreed value, the activity level of individual participants, number of messages sent, and types of failures
- used where durability is required
 - for example, to replicate a file or a database
- compared to the BGA problem, in this case we consider only crash-stop behaviour (not byzantine)
 - i.e. simpler case (non resistente ad attacchi bizantini)

Lamport, Leslie (May 1998). *"The Part-Time Parliament"*. ACM Transactions on Computer Systems. 16 (2): 133–169.

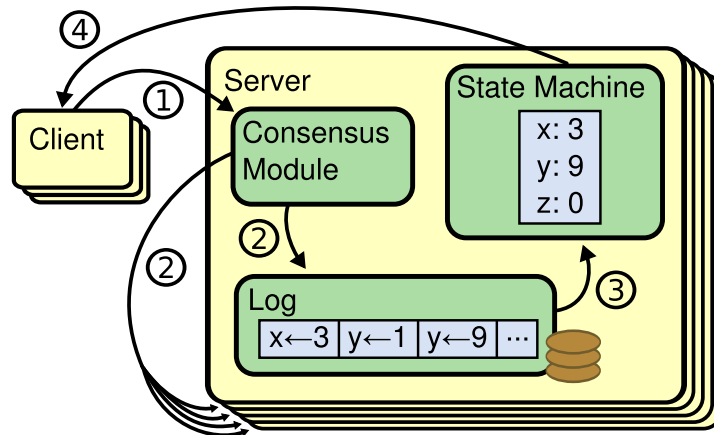
FURTHER EXPLORATIONS - RAFT

- **Raft** (Reliable, Replicated, Redundant, And Fault-Tolerant)
 - a consensus algorithm designed to be easy to understand
 - offers a generic way to distribute a state machine across a cluster of computing systems, ensuring that each node in the cluster agrees upon the same series of state transitions
 - <https://raft.github.io/>
 - it's equivalent to Paxos in fault-tolerance and performance, but easier to understand and implement
 - Open-source reference implementations available
 - Go, C++, Java, and Scala

Ongaro, Diego; Ousterhout, John (2013). "In Search of an Understandable Consensus Algorithm"

REPLICATED STATE MACHINES

- Consensus typically arises in the context of replicated state machines, a general approach to building fault-tolerant systems
- Each server has a state machine and a log
- The state machine is the component that we want to make fault-tolerant
 - e.g. a hash table



- It will appear to clients that they are interacting with a single, reliable state machine, even if a minority of the servers in the cluster fail.
- Each state machine takes as input commands from its log.
 - e.g. command: “set x to 3”

REPLICATED STATE MACHINES

- A consensus algorithm is used to agree on the commands in the servers' logs.
 - the consensus algorithm must ensure that if any state machine applies set x to 3 as the n th command, no other state machine will ever apply a different n th command.
- As a result, each state machine processes the same series of commands and thus produces the same series of results and arrives at the same series of states.